AGENTIC RL · OPENENV · RESEARCH PROPOSAL

StaffingGym

An Agentic Reinforcement Learning Environment

A novel RL benchmark where an AI agent operates a staffing agency — sourcing, interviewing, rating, and placing candidates into multi-role client projects to maximise profit.

Single Agent | Tool-based Agentic RL | LLM-in-the-loop | Real economic dynamics

THE PROBLEM

Staffing is a high-stakes balancing act

Over-Hiring

Benched candidates burn $1,538–$2,500/wk in salary with zero revenue. Cash depletes rapidly with no placement in sight.

Project Deadlines

Multi-role projects expire if unfilled. Each missed role costs the agency the full billing opportunity plus client satisfaction damage.

Multi-Role Complexity

Projects require specific developer types, seniority, and skill scores. Revenue only unlocks when ALL roles in a project are filled.

Client Relationships

Satisfaction scores decay on poor fills and slow delivery. Churned clients stop submitting projects — destroying long-term LTV.

💡 **The optimal policy** lives in a narrow band between bankruptcy from over-hiring and client churn from under-hiring. No rule-based system can reliably find it.

THE ENVIRONMENT

Three actors. One agent.

CLIENT Simulated

Submits multi-role projects per week

Pays **$100K–$162.5K/yr** per placed candidate

Satisfaction score → churn risk

Industry shapes developer type demand

STAFFING AGENCY The Agent

Observes state via `GET tools`

Acts via `EXECUTE tools`

Manages P&L, bench, placements

Goal: maximise cumulative profit

CANDIDATE Simulated

5 developer types, 3 seniority levels

Earns **$55K–$130K/yr** based on rating

Patience window — leaves if benched too long

Rated 1–5 via LLM interview

Episode: 52 steps (1 business year) · **Step:** 1 week · **Terminal:** bankruptcy (`cash < 0`) or T_max · **Seed capital:** $50,000

AGENT DECISION LOOP

Observe → Decide → Act → Reward

GET Tools
Agency state
Client projects
Candidate pool
Market demand

→

Policy π(obs)
RL agent selects action from variable action set each step

→

EXECUTE Tools
Find / Interview
Hire / Match
Let go / Pass
Negotiate

→

LLM Judges
Interview result
Project fit rating
Salary negotiation
Satisfaction update

→

Reward R(t)
Billing revenue
− Salary burn
− Costs
→ next obs

Tools (GET)	Tools (EXECUTE)	LLM Calls	Steps / Episode
7	10	5	52

MULTI-ROLE PROJECTS

Revenue is gated. All roles must be filled.

📋 **PROJECT P001 · Acme Corp**

🔷 Backend Dev × 2	✓ 1/2 filled
🔴 ML Engineer × 1	✗ 0/1 filled
🟡 DevOps × 1	✓ 1/1 filled

Fill status: **PARTIAL (2/4)** Deadline: **3 weeks**

Weekly revenue: **$0 until SEALED**

When SEALED → 4 candidates placed × client_rate_weekly = **$7,692–$12,500/wk** revenue

🎯

3-Condition Match

`developer_type` match
`skill_score ≥ min_skill`
`seniority` compatible

⚠️

Adjacent Match Penalty

e.g. fullstack → frontend
match_score: **0.7** (vs 1.0)
Reduces client satisfaction

🚫

Illegal Actions Blocked

Type mismatch or skill below min → **action masked**. Agent cannot force invalid assignments.

RATING SYSTEM & ECONOMICS

Rating drives everything. Salary. Revenue. Margin.

RATING	LABEL	CANDIDATE SALARY/YR	CLIENT RATE/YR	MARGIN/WK	BREAK-EVEN
⭐ 1.0–1.9	Poor	$55,000	$68,750	+$265	7.5 wks
⭐⭐ 2.0–2.9	Below Avg	$70,000	$87,500	+$337	5.9 wks
⭐⭐⭐ 3.0–3.9	Average	$85,000	$106,250	+$408	4.9 wks
⭐⭐⭐⭐ 4.0–4.4	Good	$105,000	$131,250	+$504	4.0 wks
⭐⭐⭐⭐⭐ 4.5–5.0	Excellent	$130,000	$162,500	+$625	3.2 wks

Fixed Markup Rule

`client_rate = salary × 1.25`

Always 25% margin

Composite Rating Formula

`0.4×base + 0.6×fit`

Project fit weighted higher

⚠️ Bench Risk

`-$2,500/wk`

Rating-5 benched candidate

REWARD FUNCTION

Every week, the agent wins or bleeds.

```
R(t) = R_billing - C_salary - C_onboarding - C_severance - C_expired
- C_churn + B_speed
```

+	Billing Revenue	Σ client_rate_weekly(c) · placed	Recurring · rating-tier based
−	Salary Burn	Σ salary_weekly(c) · hired	Placed AND benched
−	Onboarding	$2,000 per hire event	One-time, rating-independent
−	Severance	2 × salary_weekly per let_go	Higher rated = costlier to fire
−	Expired Project	client_rate × weeks_remaining	Opportunity cost
−	Client Churn	estimated LTV lost	Long-horizon strategic cost
+	Speed Bonus	10% margin if sealed < 2 wks	Optional shaping signal

LLM-IN-THE-LOOP

Where probability distributions aren't enough.

🔦 **llm_interview()**
Returns `base_rating 1–5`, red flags, and proceed signal. A Beta distribution over skill_score loses all role context.

🎯 **llm_project_fit()**
Returns `project_fit_rating` and satisfaction delta. Fit depends on client industry, team stack, and candidate background.

💰 **llm_salary_negotiation()**
Accept, reject, or counter offer. A sigmoid over salary delta ignores competing offers, urgency, and career framing.

😤 **llm_client_satisfaction()**
Updates satisfaction score after every material event. Client reaction has memory, personality, and industry norms — not a delta function.

🚶 **llm_candidate_leave()**
Per-step decision for candidates with patience ≤ 2. Whether they leave depends on competing offers, financial situation, and agency behaviour — not just a countdown timer.

⚠️ **Reproducibility:** All LLM outputs must be cached by `(episode_seed, step, call_type, input_hash)` to ensure RL trajectory replay.

RESEARCH POSITIONING

Genuinely novel. Genuinely unsolved.

What Exists

OR-based shift scheduling
Supervised candidate scoring
LLM recruiting pipelines
Marketplace matching (Uber/LinkedIn)

What's Missing

Staffing agency as RL agent with P&L reward
Multi-role sealing as delayed combinatorial reward
Dual churn pressure (supply + demand side)
Tool-use agentic loop as action space

DIMENSION	STAFFINGGYM	CLOSEST PRIOR WORK
Action space	Tool-use chains (agentic)	Discrete action tables
Reward signal	Real P&L (recurring + one-time)	Proxy task completion scores
Semantics	LLM judges (5 call sites)	Fixed probability distributions
Relationships	Client LTV + satisfaction dynamics	Stateless matching
Benchmark	Open-source Gym env (proposed)	No equivalent exists

OPEN RESEARCH QUESTIONS

The hard problems we're tackling.

ARCHITECTURE

A1 — Variable-length action space: pointer networks vs. action masking vs. bipartite matching head?

A3 — Single-step RL or multi-step tool-use chain per episode step (ReAct vs. PPO)?

A6 — Bipartite matching: end-to-end learned or constrained optimisation sub-problem?

E7 — Full observability (MDP) or partial (POMDP)? Large algorithm impact.

TRAINING

R1 — Sparse rewards from multi-role sealing: shaping, intrinsic curiosity, or hierarchical RL?

R4 — Reward hacking prevention: confirm-never-fill, hire-fire cycling, always-pass degenerate policies.

S3 — Curriculum learning: 1 client → 3 clients → full N-client stochastic env for stable cold-start.

R5 — Scalarised reward or multi-objective Pareto-front (profit vs satisfaction vs candidate welfare)?

IMPLEMENTATION ROADMAP

From spec to open-source benchmark.

PHASE 1

Foundation

- env.py skeleton
- State dataclasses
- Greedy baseline policy
- LLM stub wiring
- 1 client, 1 dev type

PHASE 2

Full Env

- Multi-role projects
- 5 developer types
- Client satisfaction
- LLM calls live
- Episodic caching

PHASE 3

RL Training

- PPO / SAC baseline
- Curriculum stages
- Action masking
- Observation encoder
- Reward shaping

PHASE 4

Benchmark

- Open-source release
- Eval metrics suite
- Leaderboard
- Paper submission
- Community RFCs

Immediate next step
Build greedy baseline

Key architectural decision
ReAct vs PPO loop

Open-source name
StaffingGym

REWARD ARCHITECTURE

One primary source. Five penalty layers.

PRIMARY SIGNAL — ADVANCE_WEEK()

```
R = R_margin + R_bench_burn + R_expiry + R_churn
```

+	Placement Margin	bill_rate_wk - salary_wk per placed
−	Bench Burn	salary_wk per benched candidate
−	Expiry Penalty	bill_rate_wk × weeks_remaining
−	Churn Penalty	estimated LTV of lost client

TOOL-LEVEL REWARDS

`interview_candidate()`	−$500
`hire_candidate()`	−$2,000
`let_go_candidate()`	−2× salary/wk
`match_candidate_to_project()`	+speed bonus

BEHAVIOURAL PENALTIES

Invalid Action

Tool returns `success: False` → immediate penalty. Teaches agent to avoid illegal calls.

Repeat Call

Same execute-tool called back-to-back → penalty + added to `core.costs`. Discourages looping.

Passive Streak

Too many consecutive GET-only turns → penalty per extra turn. Prevents analysis paralysis.

🔒 Reward Capture Pattern

```
env._last_tool_reward = float(
  res.pop("reward", 0.0)
)
# "reward" key stripped before
# agent sees tool result dict
```

Agent *never* sees the reward signal — only the training loop does.

ECONOMICS — ACTUAL IMPLEMENTATION

Continuous, not fixed tiers. Market-driven margins.

CANDIDATE SALARY GENERATION

```
skill      = 0.3 + Beta(2,2) × 0.7
base       = base_salaries[seniority]
mult       = role_multipliers[dev_type]
skill_mod  = 0.8 + skill × 0.4
variance   = uniform(0.80, 1.20)

weekly = (base × mult
          × skill_mod
          × variance) / 52
```

Result: continuous salary expectations — not discrete tiers. High-skill candidates demand proportionally more.

Market Pool Churn

Uninterviewed candidates age out weekly. Pool auto-replenishes to `config.market_pool_size`. Forces agent urgency — candidates are not infinitely available.

CLIENT BILL RATE GENERATION

```
scarcity = {
  ml_engineer: 1.15,
  devops:      1.10,
  backend:     1.05,
  fullstack:   1.02,
  frontend:    1.00,
}[dev_type]

premium  = uniform(1.30, 2.00)

bill_wk  = (base × role_mult
            × scarcity
            × premium) / 52
```

Bill rates are per-role, set at project generation. Margin = `bill_rate_wk - salary_wk`. Not a fixed 25%.

Min margin/wk	Max margin/wk
~$200	**~$1,500+**
low skill + low premium	ml_eng + high premium

⚠️ **Spec vs. Implementation gap:** The original fixed-tier salary model ($55K–$130K) was a design target. The actual code uses continuous market-driven generation. The agent must learn to estimate margins from raw salary expectations and bill rates — not look up a tier table.

TRAINING STRATEGY

Curriculum. Shaping. Degenerate policy prevention.

CURRICULUM STAGES

Stage 1 — Warm-up
1 client · 1 dev type · 1-role projects · fixed salaries

Stage 2 — Intermediate
3 clients · 3 dev types · 2-role projects · market salaries

Stage 3 — Full Env
N clients · 5 dev types · multi-role · stochastic arrivals · patience decay

DEGENERATE POLICY TRAPS

Always-GET Policy
Spam observation tools forever.
Fix: passive_streak_penalty

Confirm-Never-Fill
Lock in projects then abandon.
Fix: expiry penalty at bill_rate × weeks_remaining

Hire-Fire Cycling
Hire then immediately let go.
Fix: $2K onboard + 2× severance + repeat penalty

Always-Pass Policy
Decline all projects to avoid risk.
Fix: client churn + LTV loss

ALGORITHM & ARCHITECTURE

Baseline
Rule-based greedy: hire when demand > bench. Surprisingly hard to beat. Build first.

RL Algorithm
PPO for discrete tool selection. Or LLM-as-agent (ReAct) with GRPO/REINFORCE for policy gradient.

Obs Encoding
Fixed scalar features + attention over variable-length candidate/project sets. GNN for bipartite matching.

Reward Shaping
Speed seal bonus + behavioural penalties handle sparse multi-role sealing signal.

STAFFINGGYM

The first agentic RL benchmark for business operations.

Real economics. Real tradeoffs. Real learning.

Staffing Agency as Agent | Multi-Role Projects | LLM Judges

Real P&L Reward | Tool-based Actions | Open Source